

Nome do Projeto: Arkanis

Nome do Professor: Lucy Mari

Nome dos Integrantes: Aleff Silva, João Colombo, Luciano Reis e Matheus Zimmer

1. Introdução

O objetivo deste documento é aplicar conceitos de análise de complexidade de algoritmos e técnicas de otimização no contexto do Projeto Interdisciplinar (Flex Automation). O projeto consiste em um sistema IoT embarcado em um ESP32 que monitora variáveis ambientais (temperatura, umidade, pressão, gás e fogo) em ambientes de risco e valor, como museus.

Na versão inicial do sistema, a leitura dos sensores e a comunicação com o broker MQTT (HiveMQ) funcionavam de forma síncrona. Para garantir que o microcontrolador não perca ciclos de processamento e que o tempo de resposta em situações de emergência seja o menor possível, aplicamos a análise de complexidade assintótica (Notação Big-O) para identificar gargalos, como laços de repetição bloqueantes, e propusemos otimizações fundamentadas na teoria dos autômatos e eficiência de algoritmos.

2. Análise de Complexidade do Código Original

Antes da otimização, o código do ESP32 possuía as seguintes características de complexidade:

2.1 Complexidade de Tempo:

- **Leitura de Sensores e Máquina de Estados:** A leitura do sensor BME280, a conversão analógica do MQ-2 e o processamento lógico da máquina de estados (if/else) executam em tempo constante, ou seja, $O(1)$.
- **Laço de Reconexão (reconnectMQTT):** O código original possuía um bloco while (!mqtt.connected()) que forçava um delay(5000). Em caso de queda da rede, a complexidade de tempo do loop() passava de $O(1)$ para um pior caso de $O(2^n)$, bloqueando totalmente a execução. Durante esse bloqueio, a detecção de incêndio (fogo) não era processada.
- **Publicação de Dados:** O sistema publicava dados no MQTT a cada iteração do loop, gerando uma complexidade de comunicação linear em relação ao tempo, independentemente de haver mudança no ambiente.

2.2 Complexidade de Espaço:

- A alocação de memória para o JSON e as variáveis globais possuem tamanho fixo, resultando em uma complexidade espacial de $O(1)$. Isso é adequado para sistemas embarcados com memória limitada.

3. Técnicas e Algoritmos de Otimização Aplicados

Para resolver os gargalos temporais e de comunicação, foram aplicadas duas otimizações principais:

3.1 Transformação do Laço Bloqueante em Autômato Temporizado (Não-Bloqueante)

Removemos o algoritmo de repetição `while` com `delay()` e aplicamos uma técnica de varredura assíncrona utilizando a função `millis()`. A reconexão com a rede passa a ser tratada como uma transição de estado que só é avaliada se o tempo decorrido atingir o limite estipulado, reduzindo a complexidade do pior caso de reconexão no loop principal para $O(1)$.

3.2 Algoritmo de Compressão Semântica (Deadband/Thresholding para Envio)

Em vez de enviar os dados em todo ciclo contínuo, aplicamos uma otimização no fluxo de saída. O sistema agora memoriza o último estado enviado e só processa um novo pacote JSON MQTT se:

1. Houver uma mudança de estado (ex: de NORMAL para EMERGENCIA).
 2. A variação da temperatura/gás (Delta) for maior que um limiar pré-definido.
- Isso otimiza o uso da rede e reduz o processamento gasto na serialização do JSON.

4. Aplicação no Projeto (Entrada e Saída)

As otimizações alteraram o fluxo de processamento de entrada e saída do firmware:

4.1 Entrada (Inputs)

- **Dados Físicos ($O(1)$):** Leituras contínuas dos pinos MQ2_PIN, FOGO_PIN e barramento I2C (BME280).
- **Clock Interno ($O(1)$):** Leitura de `millis()` para cálculo assíncrono do tempo de envio e tentativas de reconexão de rede.

4.2 Processamento (Otimizado)

O laço principal (loop) foi refatorado para garantir que a leitura de detecção de fogo e gás ocorra a cada ciclo de máquina, ininterruptamente.

- *Condição de Conexão:* `if (!mqtt.connected() && (millis() - ultimaTentativa > 5000))`
`rightarrow` Tenta reconectar sem travar o processador.
- *Condição de Publicação:* `if (estadoAtual != estadoAnterior || (millis() - ultimoEnvio >`

10000)) → Evita publicações redundantes.

4.3 Saída (Outputs)

- **Atuadores Locais ($O(1)$):** LEDs de sinalização, relé de segurança e buzzer, acionados imediatamente após a avaliação da máquina de estados.
- **Rede MQTT:** Saída de dados em JSON formatado de maneira assíncrona, enviando os dados para o tópico `flex/sala_museu_1/sensor_01` apenas quando necessário ou em intervalos estipulados de "keep-alive" (ex: a cada 10 segundos).

5. Resultados Obtidos

A aplicação da análise de complexidade e das técnicas de otimização resultou em melhorias diretas no Projeto Interdisciplinar:

1. **Garantia de Tempo Real:** A máquina de estados de emergência agora opera em tempo constante estrito ($O(1)$), garantindo que o alarme de incêndio (Buzzer e Relé) soe em frações de milissegundo, mesmo se o museu ficar sem internet.
2. **Eficiência de Banda:** A técnica de *thresholding* reduziu drasticamente a quantidade de pacotes desnecessários enviados ao HiveMQ Cloud, economizando dados e processamento de backend.
3. **Robustez do Sistema:** O fim dos atrasos (delays) síncronos evitou que o microcontrolador sofresse interrupções forçadas pelo *Watchdog Timer* (WDT) interno do ESP32.

6. Observações Reflexivas

Pontos positivos:

- A compreensão da complexidade de tempo nos permitiu enxergar que códigos que parecem inofensivos (como um `while` de reconexão de rede) podem comprometer a função vital de um projeto de segurança (detectar incêndios).
- A conversão de lógicas bloqueantes para máquinas de estados temporizadas tornou o firmware altamente profissional e resiliente.

Dificuldades encontradas:

- Mudar o jeito de programar: Parar de usar pausas que travam o sistema e passar a usar o `millis()` para controlar o tempo. Isso permite que a placa faça várias tarefas ao mesmo tempo, mas exige mais atenção do programador para não perder a conta de cada uma.
- Achar a medida certa (o "delta"): Descobrir qual é a variação exata do sensor que justifica enviar uma mensagem nova. O desafio é economizar a internet sem deixar passar mudanças importantes na leitura.

Possíveis melhorias (Trabalhos Futuros):

- Aplicar estruturas de dados circulares (Fila circular) para armazenar os eventos críticos enquanto o dispositivo estiver offline ($O(n)$ de espaço), garantindo que o backend receba o histórico do que aconteceu durante a queda da internet assim que a conexão voltar.

7. Conclusão

A aplicação dos conceitos de Teoria da Computação e análise de complexidade de algoritmos se mostrou indispensável para o amadurecimento do nosso projeto IoT. Compreender que um dispositivo que monitora vidas e patrimônios (como um museu) não pode ter complexidade de processamento $O(2^n)$ em casos de falha de rede permitiu refatorar o código para uma abordagem estritamente $O(1)$ em seu laço principal de controle. O resultado é um sistema otimizado, veloz, confiável e alinhado com as melhores práticas de engenharia de software embarcado.